灵境竞赛组 · 后端 AI 方向 · 第三阶段

PCB 缺陷检测平台项目说明

系统架构、访问步骤、运行操作与项目总结

作者:杨淇超

日期:2026 年 10 月

在线访问与使用说明

1. 访问地址

页面	地址	用途
在线操作页面	http://39.107.27.28/ui/	登录、上传图片、查看两种方案结果
接口文档	http://39.107.27.28/docs	查看和测试 FastAPI 接口
健康检查	http://39.107.27.28/health	检查服务与 Redis 队列状态

在线地址只在云服务器租用期内有效。`127.0.0.1:8000` 是本机开发地址,只有先在 运行项目的电脑上启动 FastAPI 后才会生效,不是当前公网演示地址。

2. 验收复现图片下载

考核人员可以直接下载项目实际使用的输入图片:

下载地址:
https://github.com/xiaozhu659/pcb-defect-platform/raw/main/examples/input/missing_hole01.jpg

下载后的文件名应为 `missing_hole01.jpg`,大小约 1.36 MiB。 SHA-256:

```
9e05904962c65e4bd51535bca0a0a2c3c216baa98cc52360506319e959067676
```

下载后不需要修改图片,直接在网页中上传即可。该输入对应的 YOLO+SAM 和 DeepSeek 参考结果保存在:

```
examples/output/yolo_sam_result.png
examples/output/deepseek_result.png
```

视觉大模型结果可能因模型更新而变化,应以实际重新运行的结果为准。

3. 网页操作步骤

1. 使用浏览器打开 http://39.107.27.28/ui/ ,不要先访问 `/docs`。
2. 在页面顶部输入演示账号和密码,然后点击“登录”。
3. 在“上传图片”区域选择一张 JPG、PNG、BMP 或 WEBP 格式的 PCB 图片。
4. 点击“开始检测”。系统会创建任务,并把任务交给后台 Worker 执行。
5. 等待页面出现结果。首次运行需要加载模型,在 2 GiB CPU 服务器上通常需要

1 到 3 分钟;页面最长等待 15 分钟,期间不要重复刷新。

6. 查看上方文字中的任务编号、缺陷数量、类别和耗时。
7. 左侧图片展示 YOLO 检测框与 SAM 分割掩膜;

右侧图片展示 DeepSeek 判断的位置。

演示账号由部署者在本地 `.env` 文件中配置,不会提交到 GitHub。验收人员需要使用
提交者提供的演示账号,不要使用真实邮箱或其他账号密码。

4. 结果怎么阅读

- 左侧的蓝色矩形是 YOLO 检测框,红色区域是 SAM 生成的分割掩膜;

- 掩膜面积可以进一步用于评估缺陷大小;
- 传统方案会分别显示 YOLO 耗时、SAM 耗时和总耗时;
- 如果 SAM 不可用或分割失败,方案名称会明确标注“仅检测”或“分割失败”;
- 右侧的红色叉号表示视觉大模型自己判断的缺陷中心;
- 如果大模型认为没有缺陷,右侧只显示原图,上方文字会写明“无”;
- 如果大模型返回内容无法解析,会显示“大模型方案:失败”,不能解释为“0 处缺陷”;
- 两种方案结果不一致时,应以实际数据、定位误差和任务定义综合分析,

不能默认大模型的判断一定正确。

5. 常见访问问题

- 如果 `127.0.0.1:8000` 无法打开,说明本机 FastAPI 没有启动;
- 如果公网页面暂时显示 502,通常是 Web 容器正在启动,等待 1 到 2 分钟后刷新;
- 如果登录失败,检查 `.env` 中的 `APP_USER` 和 `APP_PASSWORD`;
- 如果健康检查返回的队列不是 `redis`,检查 Redis 容器或服务器网络;
- 如果任务超过 15 分钟仍未完成,查看 Worker 日志和服务器内存状态。

1. 项目目标

本项目复用第二阶段已经训练和评估过的 PCB 缺陷检测能力,把原本需要执行命令行脚本的流程做成一款可以通过浏览器操作的 Web 应用。用户上传一张 PCB 图片后,系统在同一任务中运行两套方案:

1. 传统方案:YOLO11n 定位缺陷,再把检测框作为提示交给 SAM 2.1 分割。
2. 视觉大模型方案:调用 DeepSeek 的图片输入能力,要求它返回缺陷类型、位置、严重程度和总结。

两种方案的标注结果在网页中并排展示,便于直观比较。

2. 系统架构

```
浏览器
  |
  v
nginx :80
  |
  +--> /ui/  Gradio 页面
  |
  +--> /api/ FastAPI 接口
               |
               +--> SQLite:保存任务状态和结果
               |
               +--> Redis:保存待处理任务
                        |
                        v
                    RQ Worker
                        |
                        +--> YOLO + SAM
                        +--> DeepSeek API
```

四个 Docker 容器分工如下:

容器	作用
`nginx`	对外提供 80 端口,反向代理网页和 API
`web`	FastAPI、JWT 登录、Gradio 页面、任务查询
`worker`	从 Redis 取任务,执行两套模型推理并保存结果
`redis`	消息队列,解耦网页请求与耗时推理

3. 请求流程

1. 用户在 `/ui/` 输入用户名和密码。
2. `POST /api/login` 校验账号密码并返回 JWT。
3. 用户上传图片,`POST /api/jobs` 先检查文件大小、扩展名、真实格式和像素数量,

通过后保存到 `uploads/`。

4. FastAPI 创建 SQLite 任务记录,并通过 RQ 把任务编号放入 Redis。
5. Worker 把任务状态改为 `running`,依次运行两套方案。
6. Worker 将两张结果图保存到 `outputs/`,并把结构化结果写回 SQLite。
7. Gradio 页面轮询任务状态,完成后并排展示两张结果图。

任务状态按 `queued -> running -> done/failed` 变化。任何一套方案失败时,
另一套方案的结果仍会保留,错误原因会记录在任务数据中。只有两套方案都失败时, 整体任务才标记为 `failed`。

如果任务提交 Redis 队列失败,接口会返回 503,并在 SQLite 中立即把任务标成 `failed`。`queued` 或 `running` 状态超过 `STALE_JOB_MINUTES`(默认 30 分钟)时, Web
启动或后续查询会自动把任务标记为失败,避免任务永久卡住。

4. 数据设计

SQLite 中只有一张 `jobs` 表:

字段	含义
`id`	任务编号
`filename`	原始文件名
`image_path`	上传图片的服务器路径
`status`	排队、运行、完成或失败
`created_at` / `finished_at`	创建和结束时间
`yolo_result` / `llm_result`	两种方案的 JSON 结果
`yolo_ms` / `llm_ms`	传统方案总耗时(YOLO + SAM) / 大模型方案耗时
`error`	整体任务错误信息

权重、数据库、上传图片和结果图片都不会提交到 Git,避免仓库体积失控和密钥泄漏。

5. 身份认证

`POST /api/login` 使用环境变量中的账号密码进行认证,成功后签发 HS256 JWT。
除健康检查、登录和页面本身外,任务提交、查询和结果图片接口都要求:

```
Authorization: Bearer <token>
```

登录接口会返回如下结构:

json

```
{
  "access_token": "eyJ...",
  "token_type": "bearer"
}
```

`access_token` 是项目为本次登录生成的临时凭证,不是 GitHub token,也不是 DeepSeek API Key。默认有效期为 12 小时,过期后重新登录即可。网页会自动保存 token; 在 `/docs` 中手动测试时,需要先执行 `/api/login`,再把返回值填入“Authorize”。

当前实现用于教学和竞赛演示,账号密码存放在环境变量中。正式生产系统还应把密码改为哈希存储,并增加用户表、权限模型、HTTPS 和 token 吊销机制。

6. 部署结果

2026-10-22 已在 Ubuntu 22.04 云服务器完成部署和测试:

- 2 vCPU、2 GiB 内存、40 GiB 磁盘;
- Docker、Docker Compose 和 2 GiB Swap;
- 四个容器全部正常运行;
- `/health` 返回 `{"status":"ok","queue":"redis"}`;

- 未登录访问任务接口返回 401;
- 损坏图片会被上传校验拦截并返回 400;
- JWT 登录和 Redis 队列任务执行成功;
- 示例图检出 3 处缺孔,YOLO + SAM 耗时约 9.2 秒;
- DeepSeek 返回结构化结果,两张结果图均能正常下载。

服务器的 CPU 与内存有限,因此 SAM 使用 256 输入,Worker 设置 1150 MiB 内存上限,并限制数学库线程数。这样即使推理占用较多资源,网页和 API 仍保持可访问。

7. 项目创新点

1. 同一张图片直接并排展示传统模型与视觉大模型的判断,差异一目了然。
2. YOLO 负责定位、SAM 负责精细分割,能够计算缺陷面积,补足大模型只能给文字描述的不足。
3. 使用 Redis + RQ 把耗时推理从 HTTP 请求中拆出,避免网页长时间阻塞。
4. 两套方案相互独立,其中一套失败不会覆盖另一套的可用结果。
5. 用 Docker Compose 把 Web、Worker、Redis 和 nginx 一次编排,便于复现和迁移。

项目还包含上传校验单元测试和端到端冒烟测试,可自动检查健康接口、页面、JWT、 任务队列和两张结果图。

8. 个人总结

刚开始做第三阶段时,我以为主要工作只是把第二阶段的模型接到一个网页上。真正开始后才发现,从命令行脚本到一个稳定可用的平台,中间还有很多工程问题:浏览器的请求怎样交给后台、任务状态怎样保存、两个容器怎样通过 Redis 通信、JWT 怎样保护接口,以及服务器内存不足时怎样保证页面不被拖垮。

这次最耗时间的问题发生在云服务器首次推理。服务器只有 2 GiB 内存,模型加载后网页一度响应很慢。后来我通过查看进程和容器状态定位到资源竞争,增加了 Swap,限制 Worker 内存与线程数,并把 SAM 输入从 1024 调整为 256。再次测试后,YOLO+SAM 和 DeepSeek 都能完成,网页也保持可访问。这让我认识到,模型指标只是项目的一部分,部署环境、失败处理和服务稳定性同样决定一个 AI 系统能不能真正交付。

AI 辅助明显提高了写代码和排查问题的速度,但我不能只接受生成结果。每一条接口、每一个容器和每一个参数,最终都需要自己能解释为什么这样设计。通过这个项目,我对"训练一个模型"和"交付一个 AI 应用"之间的区别有了更具体的理解。

如果继续完善,我会增加用户表与密码哈希、HTTPS、多 Worker 和更完整的监控;模型方向则继续补迁移学习对照实验,并尝试量化后在嵌入式设备上部署。

9. 已知限制

- SQLite 适合单机演示,多实例部署时应替换为 PostgreSQL 等数据库。
- 单 Worker 串行处理任务,高并发场景应增加 Worker 或独立推理服务。
- 当前演示使用 HTTP,正式上线应配置域名和 HTTPS。
- DeepSeek 判断受图片、提示词和模型随机性影响,页面保留原始输出,不作为绝对结论。

10. 项目资料索引

- 在线操作页面:http://39.107.27.28/ui/
- 接口文档:http://39.107.27.28/docs
- 健康检查:http://39.107.27.28/health
- 代码仓库:https://github.com/xiaozhu659/pcb-defect-platform
- 第二阶段项目:https://github.com/xiaozhu659/pcb-defect-comparison

仓库内的 `docs/部署与验收.md` 说明如何从零启动和检查项目, `docs/验收记录.md`
记录实际测试结果,视频录制稿与个人感受保存在本地学习资料目录, 不提交到公开代码仓库。